

# Harness Engineering

## 长任务为什么需要 Harness

任务需要连续执行 → 每一步都会产生新的结果和状态 → 后续决策依赖这些结果 → 系统需要检查、记录并纠正整个过程。

大模型可以根据当前输入生成回答，也可以提出下一步的工具调用。但一个任务能否完成，还取决于工具是否执行成功、返回结果是否可信、中间状态是否保留，以及最后有没有明确的验收标准。

具体示例：

让 Agent 开发一个支持 CSV 导入的数据看板。用户上传文件后，页面需要显示数据预览、按日期筛选，并将筛选结果导出为 CSV。

模型可能很快写出界面，但实际运行后仍可能出现问题：上传按钮没有连接处理逻辑；页面显示了筛选条件，导出的却是全部数据；刷新页面后，刚才的操作状态丢失。

这些问题分布在执行链路的不同位置。若系统只检查最终回复是否写了“已完成”，就无法区分界面已经生成与功能已经通过验证。

Harness 的作用是组织模型外部的运行过程，让任务有状态、有反馈，也有明确的结束条件。

## Prompt、Context 与 Harness

Prompt Engineering 关注任务如何表达。角色、目标、约束、示例和输出格式，都属于这一部分。

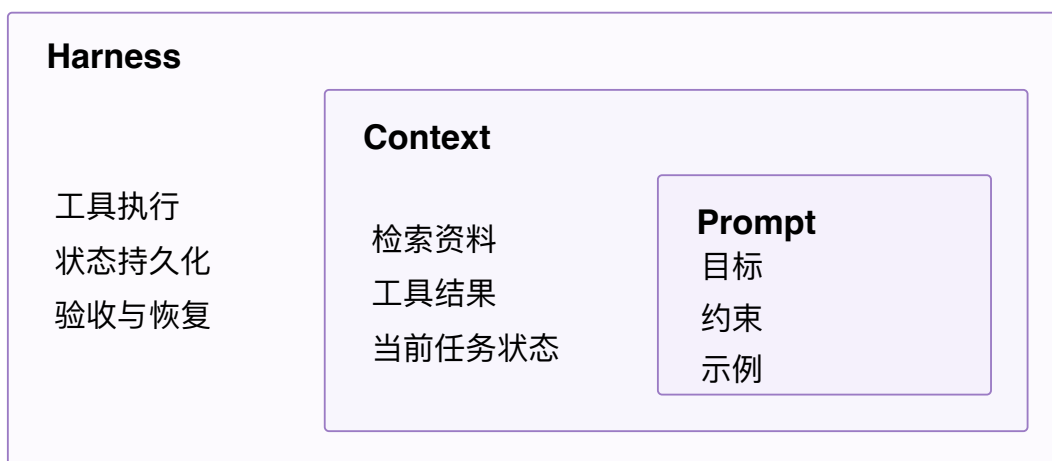
例如，对上面的看板任务补充：“日期采用 YYYY-MM-DD；空文件需要提示；导出的内容必须与当前筛选结果一致。”这些指令帮助模型理解目标，但 CSV 的实际字段、现有代码结构和项目依赖，仍需要系统提供。

Context Engineering 关注模型在当前一步能看到什么。它包括任务指令，也包括检索资料、工具返回、历史对话、当前状态和中间产物。其作用是把当前决策需要的信息组织进上下文。

Harness Engineering 的范围进一步扩展到整个运行系统：如何调用工具，何时继续执行，怎样保存状态，如何验收，以及失败后怎样恢复。

图 1 按关注范围划分边界：Prompt 位于 Context 内，Context 的组织又属于 Harness 的一部分。这种划分便于分析系统，实际实现中的职责可能交叉。

Agent 的系统组成可以简写为  $\text{Agent} = \text{Model} + \text{Harness}$ 。这里的加号是系统组成的示意，不能理解为性能公式。模型能力、运行环境与任务难度会共同影响结果。



概念范围示意；实际系统中的边界可能重叠。

图 1：Prompt、Context 与 Harness 的关注范围

## 整体运行流程

一次任务可以按下面的过程组织：读取目标与已有状态 → 构造当前上下文 → 决定下一步动作 → 执行工具 → 检查实际结果 → 更新状态，再继续下一轮。

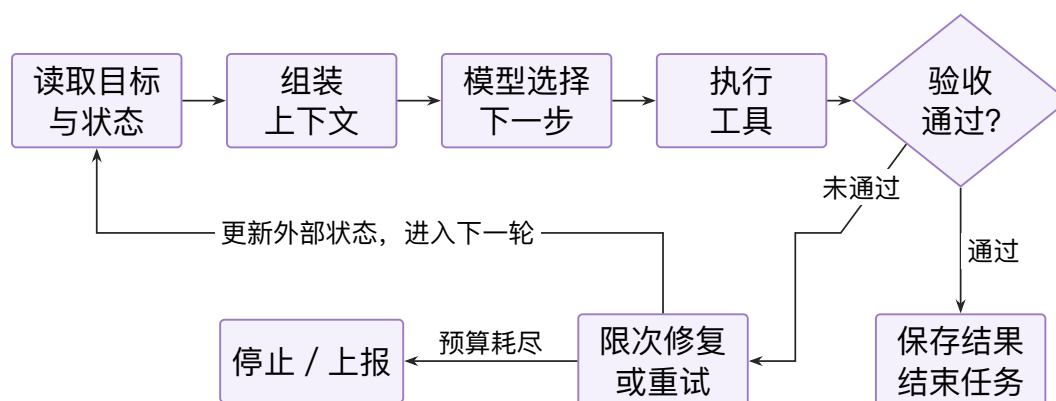


图 2：带验收、状态更新和退出条件的执行闭环

其中，工具执行成功与任务验收通过是两件事。文件写入成功，只能说明产物已经落盘；浏览器页面能够打开，只能说明应用能够启动。导出结果是否满足要求，还需要检查文件内容。

若检查未通过，系统应将失败证据送回下一轮上下文。例如，记录触发问题的输入、预期输出、实际输出和错误日志，再让 Agent 修复。

若重试次数、时间或成本达到预算，系统需要停止并保留现场。否则，“生成 → 检查 → 修复”的闭环也可能变成无休止的重复。

下面是机制示意，不对应某个框架的具体 API：

读取目标、验收条件与已保存状态  
在预算内循环：  
    组装当前步骤需要的上下文

由模型提出动作，经约束检查后执行  
检查结果，保存证据并更新任务状态  
全部验收通过则结束；否则依据失败原因继续  
预算耗尽或遇到阻塞：保存现场，报告未完成项

## Harness 的构成

可以从六个方面检查 Harness 的设计：上下文、工具、执行编排、记忆与状态、评估与观测、约束与恢复。

### Context：选择当前需要的信息

首先明确任务目标与成功标准，再选择当前步骤需要的证据。固定规则、当前任务、运行状态和外部资料应当有清楚的组织方式。

例如，处理日期筛选时，模型需要看到日期字段的定义、相关组件和验收用例。此前搜索得到的几十篇无关网页，没有必要继续占据上下文。

RAG 是补充信息的一种方法。文档先切块并建立索引，查询后再筛选、排序检索结果，将相关内容放入上下文。历史对话何时摘要、工具输出保留多少、子任务返回原文还是结构化结论，也属于 Context Engineering。

Agent Skills 通过渐进式披露控制信息加载：先提供名称和简短描述，匹配任务后再读取具体的 SKILL.md，需要时继续加载参考资料或执行脚本。这样可以将说明书按需送入上下文。

上下文组织的目标是让当前决策所需的信息容易被找到和使用。保留什么、何时加载、如何更新，比单纯增加上下文长度更值得检查。

### Tools：连接动作与真实环境

工具层负责把模型提出的动作变成实际操作，例如读取文件、查询数据、运行代码或打开浏览器。

首先按任务选择足够用的工具集，再明确调用时机：缺少事实依据时先查证，需要确认功能时运行测试，需要检查交互时打开浏览器。

工具的能力边界、输入参数和返回结果也需要一起设计。若日期筛选测试失败，工具应返回具体断言和实际值，帮助下一轮定位问题。

工具输出也需要控制范围。完整日志可以保存在外部文件中，当前上下文只加载相关错误、必要上下文和原始文件的位置。这样既能继续分析，也保留了追查依据。

## Orchestration：组织步骤与分支

执行编排负责决定步骤之间的衔接。信息不足时先补资料；前置条件满足后再执行；结果不符合要求时进入修复或退出分支。

看板任务可以先实现数据读取，再处理筛选，最后验证导出。若 CSV 解析尚未通过，继续装饰页面就会让系统积累更多未经确认的工作。

编排也不等于把所有任务固定成同一张流程图。能由程序稳定判断的条件，可以写成明确分支；需要分析和选择的部分，再交给模型。若一个模型已经能可靠处理某段任务，额外拆分也可能增加交接开销。

## Memory and State：记录做到哪一步

当前任务状态、会话中的中间结果、长期偏好，应当分开保存。

当前任务状态回答“哪些功能已通过，哪些仍失败”；中间结果保存文件路径、已确认结论和待解决问题；长期偏好则记录跨任务仍然适用的要求，例如用户希望文章采用什么写作风格。

具体示例：

已确认：CSV 读取与数据预览通过测试  
未通过：导出文件仍包含筛选范围之外的记录  
证据：测试输出与实际导出文件的位置  
下一步：检查导出函数是否读取当前筛选状态

若这些信息只停留在一段很长的对话中，切换会话时容易丢失。将它们写入外部进度记录，新会话就可以从已验证的状态继续工作。

## Evaluation and Observability：建立验收依据

评估负责判断结果是否满足要求，观测负责提供定位问题所需的证据。

对于看板，验收可以包含：用已知输入检查筛选结果，实际点击导出按钮，再读取导出的 CSV。测试结果、页面截图、运行日志和关键指标，共同构成判断依据。

若涉及界面体验或产品完整度，可以让单独的 Evaluator 按明确标准检查。独立评估有助于减少自评偏差，但评估者仍可能漏检，因此还需要校准标准，并尽量结合可复现的检查。

观测的作用还包括区分失败原因：资料没找到、工具超时、模型误解、状态没有更新，需要采取的修复方式各不相同。

一次任务中的“检查 → 修复”，解决当前产物的问题。Harness 本身也需要改进闭环：保留执行记录 → 定位失败 → 将失败变成评测任务 → 修改系统 → 重新比较结果。沿执行记录改进 Harness 的关键，是让每次改动对应一个能够复现的问题。

归因时先检查模型实际收到的信息、工具返回的结果、环境前提与验收规则。缺失字段定义、工具超时、错误决策和评分器误判，都可能表现为任务失败，却需要不同的修复。执行记录应保存这些

可检查的输入、动作与结果，不能只留下最终回答。

比较改动时，尽量固定模型版本、任务集、初始环境与预算，并记录 Harness 版本。对有随机性的任务重复运行，同时比较完成率、耗时与成本。修复导出问题后，还要检查原本通过的上传和筛选是否退化。Agent 评测应以实际结果为主要依据；除非动作顺序本身属于业务约束，不必要求 Agent 走唯一一条工具调用路径。

**Constraints and Recovery：限制影响并恢复执行**

约束需要落实到执行边界，例如允许访问的文件范围、可调用的工具和执行预算。仅在提示词中要求“不要误操作”，不足以代替实际的权限和参数检查。

恢复则根据失败类型选择路径。临时超时可以在预算内重试；输入格式错误需要修正输入；实现破坏了已有功能，可以回到已保存的稳定版本。

对可能产生重复副作用的动作，还需要先确认上一次是否已成功执行，再决定是否重试。否则，一次网络超时可能被错误地当成“操作完全没有发生”。

**长任务的上下文交接**

状态记录还有一个具体用途：支持上下文交接。Context Reset 是长任务的一种交接方式：结束当前上下文，将工作交给一个新的会话。

其前提是先保存足够的交接材料。新会话需要知道任务目标、已完成的工作、验收证据、未解决的问题，以及下一步应该从哪里开始。

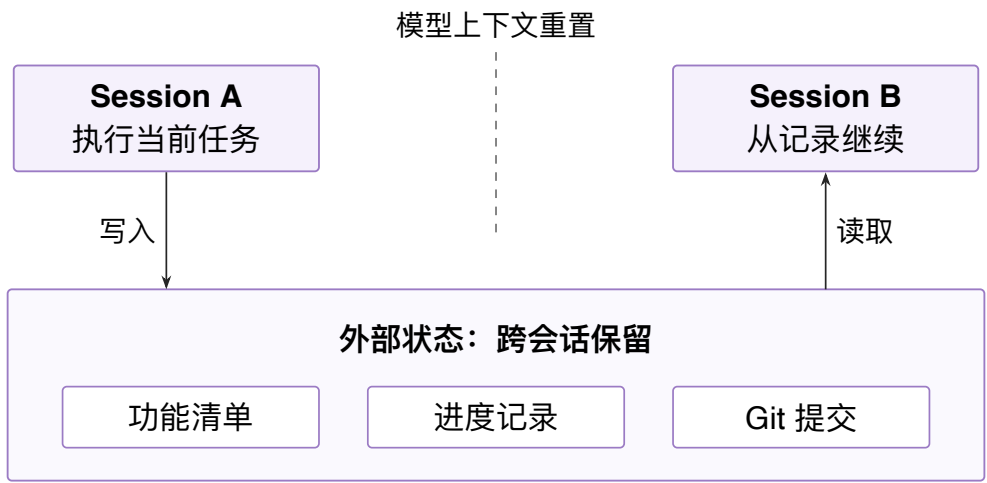


图 3：会话上下文与外部持久状态的交接

Anthropic 的长任务方案用功能清单、进度日志和 Git 记录连接多个会话。后续会话读取记录，完成一项功能，验证后再更新记录。

这里需要区分 Compaction 与 Reset。Compaction 对已有历史进行压缩，保留缩短后的连续上下文；Reset 则从新上下文开始，依赖外部交接材料恢复任务。

具体采用哪一种，需要结合模型与任务验证。在 Anthropic 的应用开发实验中，使用 Sonnet 4.5 的方案需要 Reset；切换到 Opus 4.5 后，则改为连续会话和自动 Compaction。Reset 的必要性会随模型能力和任务变化。

交接机制的作用是保留已确认的任务状态，同时控制下一轮需要加载的信息。若交接材料漏掉失败条件，即使新上下文很干净，也可能重新犯同一个错误。

进度记录还应指向具体产物与证据，例如提交版本、测试命令和失败输入。记录中的“已通过”有适用条件：代码、依赖或服务状态变化后，旧结论未必仍然成立。新会话应先读取记录，再核对当前文件与运行环境，做必要的基本验证，随后继续未完成项。摘要帮助定位现场，当前环境中的结果支持判断。

## 业界实践

### Anthropic：将实现与验收分开

Anthropic 在应用开发实验中设置了 Planner、Generator、Evaluator。Planner 将需求展开为规格，Generator 实现功能，Evaluator 在实际环境中检查交互与结果。

案例包括 2D 游戏制作工具和浏览器中的数字音频工作站（DAW）。验收者需要实际操作时间轴、检查数据读写。失败反馈具体到触发方式和预期行为，才能进入下一轮修复。

在使用 Opus 4.6 的版本中，编排进一步简化：取消 Sprint 划分，将评估移到构建末尾。生成的应用已有可用的核心功能，仍有需要修复的问题。Harness 需要随模型能力调整，其复杂度也需要通过任务效果来判断。

### OpenAI：让 Agent 能读取工程环境

OpenAI 的工程实践将简短的 AGENTS.md 用作入口，把架构、计划和规范放到可继续检索的仓库文档中。每个 Git worktree 有独立可运行的应用和观测环境，让 Agent 能操作浏览器、读取日志和指标。

架构规则也进入自动检查：通过 lint 和结构测试限制模块依赖，将违规位置和修复指引返回给 Agent。这样，约定既有文档解释，也有执行时的反馈。

对前面的看板任务，可以借鉴这一组织方式：入口说明提供项目结构和验收命令；详细文档解释 CSV 字段和交互约定；测试失败后，反馈给出相关规则与定位信息。

OpenAI 在一个内部 Beta 产品实验中构建了约百万行的仓库，其中包含代码、文档和工具。团队估计，其开发耗时约为人工开发的十分之一。人类仍负责目标、验收和反馈，不能据此推断任意项目都能获得同样的交付效率。

## LangChain：沿执行记录查找失败原因

LangChain 在固定 gpt-5.2-codex 的情况下改进系统提示、工具和执行控制，使 Terminal-Bench 2.0 评测成绩从 52.8% 提升到 66.5%，增加 13.7 个百分点，在当时的榜单上从 30 名之外进入前五。这些结果对应这次评测的任务集与运行条件。

它提供的可复用思路是查看执行记录：Agent 是否提前结束，是否陷入重复尝试，是否留下了足够的验证时间，再针对具体失败修改 Harness。

若问题出在导出验证被跳过，增加一个明确的完成前检查，比继续要求模型“认真检查”更容易验证效果。改动是否有效，还需要回到同一组任务上比较。

## 为什么这些工作需要一个名字

工具接入、业务规则梳理、状态保存、测试与失败恢复，并不是有了 Harness 这个名称之后才出现的工作。命名的一个作用，是将这些分散的工程工作放到同一个讨论框架中。

这也有组织沟通上的价值。若管理者将模型能力直接等同于产品能力，就容易低估接入业务和验证结果所需的工作。一个明确的名称，可以帮助解释时间花在哪里、需要哪些资源，以及为什么演示成功还不等于能够交付。从这个角度看，它也承担了“向上管理”的作用：校准预期，让工程投入更容易被看见。

但名称本身不能证明工作有价值。最后仍要落实到具体问题：补上了什么能力，减少了哪类失败，怎样验证效果。术语应当帮助解释工作，而不是代替解释。

## 总结

- (1) Prompt、Context 与 Harness 的关注范围逐步扩大。任务指令说明目标，上下文提供当前需要的信息，Harness 组织执行、状态与反馈。
- (2) 完整任务需要以实际结果验收。每轮动作之后检查证据、更新状态，失败时保留原因，并为重试和退出设置边界。
- (3) Harness 应围绕已观察到的失败设计。先补上缺失的验证、状态或恢复能力，再通过任务表现判断是否需要更多编排；模型更新后，也应重新检查这些结构是否仍然必要。